

INSTALLATION

Title: Sequential Deep Learning Models for Personalized Fashion Recommendation

Project Overview: This project develops a sequential recommendation system for the fashion and beauty lifestyle domain. It features an Optimized BERT4Rec model (enhanced with category-level embeddings) and SASRec/BERT4Rec baselines. Additionally, an NCF model is integrated to predict user ratings. The final stage is demonstrated through an interactive prototype for visualizing top-K recommendations and predicted rating.

System Requirements

Python version: 3.13.5

Streamlit: 1.53.1

PyTorch: 2.10.0 (Used for loading the BERT4Rec and NCF .pth weights).

Core Libraries: pandas, numpy, scikit-learn (latest stable versions via requirements.txt).

Installation & Running the Demo

Step 1: Open your terminal or command prompt and move into the demo folder:

```
cd "ITDSIU21058_LeHuynhNhaNguyen_Code/Code Demo UI"
```

Step 2: System Requirements & Environment Setup

- Create a virtual environment

```
python -m venv venv
```

- Activate the environment

```
Windows: .\venv\Scripts\activate  
  
macOS/Linux: source venv/bin/activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Run the Streamlit Application

```
streamlit run appfinal.py
```

Step 5: Access the Interface

Once the command is executed, the terminal will display a local URL. The app should automatically open in your default browser. If not, manually navigate to: <http://localhost:8501>

Note: The model weights (Bert4Rec.pth, best_ncf_model.pth) are already included in the Model/ directory. No additional training is needed to run the demo. Datasets: The necessary CSV files for the demo are located in the Data/ directory.

Demo Application Description

The demo application is designed to illustrate the inference capability and prediction accuracy of the proposed sequential recommendation system, combining a sequential model and a Neural Collaborative Filtering (NCF) rating predictor. At the beginning, three representative test cases are prepared to visually demonstrate the correctness and consistency of the model predictions.

The demo workflow operates as follows:

- The user inputs or selects a user ID from the dataset.

- The system retrieves and displays the historical purchase sequence associated with the selected user.
- This historical interaction sequence is used as the input sequence for the sequential recommendation model.
- The user can specify the number of recommended items (Top-K, e.g., Top-5, Top-10, or Top-15).
- After clicking the *Get Recommendation* button, the system generates a Top-K list of predicted next items that the user is likely to purchase.
- For each recommended item, the NCF model predicts and displays the corresponding rating score, providing an additional quantitative evaluation of user preference.

This demo is designed as a dynamic and interactive system. When the user provides new input data, the model immediately produces new prediction results without requiring retraining. In addition to using the original historical sequence, the demo also allows the user to:

- Search and select items,
- Construct a completely new interaction sequence for the selected user, and
- Generate a new Top-K recommendation list based on the newly created sequence.

This functionality demonstrates the flexibility and generalization ability of the trained sequential model when handling both real historical data and user-defined input sequences.

Model Training and Usage

In this project, the BERT4Rec and Neural Collaborative Filtering (NCF) models were trained and the resulting optimal parameters were saved as checkpoint files in .pth format. These trained models are subsequently loaded and used directly in the demo application for inference.

In addition to the optimized BERT4Rec and NCF models used in the demo, the original BERT4Rec and SASRec models were also trained as baseline methods for evaluation purposes. The evaluation results of these baseline models were recorded and stored in log files for comparison.

All models were implemented in Python using the PyTorch deep learning framework. The training process was conducted in the Google Colaboratory environment, utilizing an NVIDIA T4 GPU with 16 GB of memory and CUDA support for computational acceleration.

Model Saving and Inference

After training, the optimal model parameters are saved as checkpoint files (.pth) and later loaded into the demo application for inference. All models provided in this project are pre-trained, and retraining is optional. The demo application directly uses the saved model weights and does not require executing the training process.